```
>>> y.grad_fn
None
```

We create two trainable parameters, `m` and `b`, by setting their `requires_grad` attribute to `True`, which means PyTorch will consider these parameters as nodes in a computational graph and will store their history of computations. Any new tensors that are created using `m` and `b`, such as `y` in this case, will also have `requires_grad` set to `True` and thus will also keep a memory of their computation history. You can see that the first time we call the linear model and print `y`, it gives us a tensor with the numeric result and also shows an attribute, `grad_fn=<AddBackward0>`. We can also see this attribute directly by printing `y.grad_fn`. This shows that this tensor was created by the addition operation. It is called `AddBackward` because it actually stores the derivative of the addition function.

If you call this function given one input, it returns two outputs, like the opposite of addition, which takes two inputs and returns one output. Since our addition function is a function of two variables, there is a partial derivative with respect to the first input and a partial derivative with respect to the second input. The partial derivative of $y = a + b$ with respect to m is $\frac{\partial y}{\partial a} = 1$ and $\frac{\partial y}{\partial b} = 1$. Or if $y = a \times b$ then $\frac{\partial y}{\partial a} = b$ and $\frac{\partial y}{\partial b} = a$. These are just the basic rules of taking derivatives. When we backpropagate from a given node, we need it to return all the partial derivatives, so that is why the `AddBackward0` gradient function returns two outputs.

We can verify that PyTorch is indeed computing gradients as expected by calling the backward method on `y`:

```
>>> y = linear_model(torch.Tensor([4.]), m,b)
>>> y.backward()
>>> m.grad
tensor([4.])
>>> b.grad
tensor([1.])
```

This is exactly what we would get from computing these simple partial derivatives in our head or on paper. In order to backpropagate efficiently,